1.Salary transformation system

```java
import java.util.Arrays;
import java.util.Scanner;
import java.util.stream.Collectors;

public class SalaryTransformation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();

        long[] salaries = new long[n];
        for (int i = 0; i < n; i++) {
            salaries[i] = scanner.nextLong();
        }

        String updatedSalaries = Arrays.stream(salaries)
                .map(salary -> (long) (salary * 1.10))
                .mapToObj(String::valueOf)
                .collect(Collectors.joining(" "));

        System.out.println(updatedSalaries);

        scanner.close();
    }
}
```

2.Real-time stream analytics engine

```
import java.util.*;
import java.util.stream.*;

class Sensor {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();

      Map<String, List<Double>> data = new HashMap<>();

      for (int i = 0; i < n; i++) {
```

```
            String id = sc.next();
            double temp = sc.nextDouble();

            data.computeIfAbsent(id, k -> new ArrayList<>())
                .add(temp);
        }

        data.entrySet().stream()
            .map(e -> new AbstractMap.SimpleEntry<>(
                e.getKey(),
                e.getValue().stream()
                    .mapToDouble(Double::doubleValue)
                    .average()
                    .orElse(0.0)
            ))
            .sorted((a, b) ->
                Double.compare(b.getValue(), a.getValue()))
            .forEach(e ->
                System.out.println(e.getKey() + " " + e.getValue()));
    }
}
```

3.Maximum Profit Analyzer:

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int currentSum = arr[0];
        int maxSum = arr[0];

        for (int i = 1; i < n; i++) {
            currentSum = Math.max(arr[i], currentSum + arr[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
```

```
        System.out.println(maxSum);
        sc.close();
    }
}
```

4.Intelligent DNA pattern search

```
import java.util.*;

class Main {

    static void KMPSearch(String text, String pattern) {
        int n = text.length();
        int m = pattern.length();

        int[] lps = new int[m];
        computeLPS(pattern, lps);

        int i = 0, j = 0;
        boolean found = false;

        while (i < n) {
            if (text.charAt(i) == pattern.charAt(j)) {
                i++;
                j++;
            }

            if (j == m) {
                if (found) System.out.print(" ");
                System.out.print(i - j);
                found = true;
                j = lps[j - 1];
            } else if (i < n &&
                    text.charAt(i) != pattern.charAt(j)) {
                if (j != 0) {
                    j = lps[j - 1];
                } else {
                    i++;
                }
            }
        }

        if (!found) {
            System.out.println("-1");
        } else {
```

```
            System.out.println();
        }
    }

    static void computeLPS(String pattern, int[] lps) {
        int len = 0;
        int i = 1;

        while (i < pattern.length()) {
            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String T = sc.nextLine();
        String P = sc.nextLine();

        KMPSearch(T, P);
        sc.close();
    }
}
```

5.Banking Transaction system

```
import java.util.Scanner;

public class BankAccount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
```

```
        int n = sc.nextInt();
        long[] arr = new long[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextLong();
        }

        // Initialize with the first element, NOT 0
        long maxSoFar = arr[0];
        long currentMax = arr[0];

        for (int i = 1; i < n; i++) {
            currentMax = Math.max(arr[i], currentMax + arr[i]);
            maxSoFar = Math.max(maxSoFar, currentMax);
        }

        System.out.println(maxSoFar);
        sc.close();
    }
}
```

6.Ride Sharing Platform simulator

```
import java.util.*;

class Driver {
   String name;

   Driver(String name) {
      this.name = name;
   }
}

class Rider {
   String name;

   Rider(String name) {
      this.name = name;
   }
}

abstract class Vehicle {
```

```
    abstract double calculateFare(double distance);
}

class Bike extends Vehicle {
    double calculateFare(double distance) {
        return distance * 5;
    }
}

class Auto extends Vehicle {
    double calculateFare(double distance) {
        return distance * 12;
    }
}

class Cab extends Vehicle {
    double calculateFare(double distance) {
        return distance * 12;
    }
}

class Trip {
    Rider rider;
    Driver driver;
    Vehicle vehicle;
    double distance;

    Trip(Rider rider, Driver driver,
         Vehicle vehicle, double distance) {
        if (rider == null || driver == null ||
            vehicle == null || distance <= 0) {
            throw new IllegalArgumentException("Invalid booking");
        }

        this.rider = rider;
        this.driver = driver;
        this.vehicle = vehicle;
        this.distance = distance;
    }

    double getFare() {
        return vehicle.calculateFare(distance);
    }
}
```

```
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            String type = sc.next();
            double distance = sc.nextDouble();

            try {
                Vehicle vehicle;

                switch (type) {
                    case "Bike":
                        vehicle = new Bike();
                        break;
                    case "Auto":
                        vehicle = new Auto();
                        break;
                    case "Cab":
                        vehicle = new Cab();
                        break;
                    default:
                        throw new IllegalArgumentException(
                            "Invalid ride type");
                }

                Rider rider = new Rider("Rider" + (i + 1));
                Driver driver = new Driver("Driver" + (i + 1));

                Trip trip = new Trip(
                    rider, driver, vehicle, distance);

                System.out.println((int) trip.getFare());

            } catch (IllegalArgumentException e) {
                System.out.println("Invalid booking");
            }
        }

        sc.close();
    }
}
```

Task 7:Hashtag Frequency Counter

```java
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Scanner;

public class FrequencyCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();

        // LinkedHashMap preserves the insertion order of unique hashtags
        Map<String, Integer> counts = new LinkedHashMap<>();

        for (int i = 0; i < n; i++) {
            String hashtag = sc.next();
            counts.put(hashtag, counts.getOrDefault(hashtag, 0) + 1);
        }

        for (Map.Entry<String, Integer> entry : counts.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());
        }

        sc.close();
    }
}
```

Task 8:Smart City application

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();
      int m = sc.nextInt();

      ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

      for (int i = 0; i <= n; i++) {
```

```
            graph.add(new ArrayList<>());
        }

        for (int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        int source = sc.nextInt();
        int destination = sc.nextInt();

        boolean[] visited = new boolean[n + 1];
        Queue<Integer> queue = new LinkedList<>();

        queue.add(source);
        visited[source] = true;

        while (!queue.isEmpty()) {
            int current = queue.poll();

            if (current == destination) {
                System.out.println("YES");
                sc.close();
                return;
            }

            for (int neighbor : graph.get(current)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.add(neighbor);
                }
            }
        }

        System.out.println("NO");
        sc.close();
    }
}
```

Task 9:Calculator unit testing

```
import java.util.Scanner;
```

```
class Calculator {
    public static double evaluate(double a, String op, double b) throws
ArithmeticException, IllegalArgumentException {
        switch (op) {
            case "+":
                return a + b;
            case "-":
                return a - b;
            case "*":
                return a * b;
            case "/":
                if (b == 0) {
                    throw new ArithmeticException("Division by zero");
                }
                return a / b;
            default:
                throw new IllegalArgumentException("Invalid operator: " +
op);
        }
    }
}

public class Testing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNext()) return;

        try {
            double operand1 = sc.nextDouble();
            String operator = sc.next();
            double operand2 = sc.nextDouble();

            // Perform operation
            double result = Calculator.evaluate(operand1, operator,
operand2);

            // Simple validation assertion check
            System.out.println("Test Passed");
        } catch (Exception e) {
```

```
            System.out.println("Test Failed");
        }

        sc.close();
    }
}
```

Task 10:Secure Authentication Stress testing

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.util.StringTokenizer;

public class SecureAuthStressTest {

    public static void main(String[] args) throws IOException {
        FastScanner scanner = new FastScanner();

        Integer N = scanner.nextInt();
        if (N == null) {
            return;
        }

        StringBuilder output = new StringBuilder();

        for (int i = 0; i < N; i++) {
            String username = scanner.next();
            String password = scanner.next();

            if (isValidAuth(username, password)) {
                output.append("SUCCESS\n");
            } else {
                output.append("FAILURE\n");
            }
        }

        System.out.print(output);
    }

    private static boolean isValidAuth(String username, String password) {
        if (username == null || password == null) {
```

```
            return false;
        }

        int uLength = username.length();
        int pLength = password.length();

        // Validate boundary values: Username (3-20), Password (6-20)
        boolean validUsername = (uLength >= 3 && uLength <= 20);
        boolean validPassword = (pLength >= 6 && pLength <= 20);

        return validUsername && validPassword;
    }

    // Fast I/O to handle thousands of login attempts efficiently
    static class FastScanner {
        private final BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));
        private StringTokenizer tokenizer;

        public String next() {
            while (tokenizer == null || !tokenizer.hasMoreTokens()) {
                try {
                    String line = reader.readLine();
                    if (line == null) return null;
                    tokenizer = new StringTokenizer(line);
                } catch (IOException e) {
                    return null;
                }
            }
            return tokenizer.nextToken();
        }

        public Integer nextInt() {
            String str = next();
            return str == null ? null : Integer.parseInt(str);
        }
    }
}
```